

Title: Concurrent and Parallel Programming for K-Means Clustering Algorithm**A. Introduction**

K-Means clustering is a fundamental machine learning algorithm used in many scientific, engineering, and data analysis applications for unsupervised learning. While the sequential implementation of K-Means clustering is straightforward, it becomes computationally expensive for large datasets with high dimensionality, which are common for data science and machine learning applications.

This project explores how concurrent and parallel programming techniques in Java can be used to optimise the performance of K-Means clustering by leveraging multiple cores in modern processors.

B. Purpose

The purpose of this group project is to challenge students to deepen their understanding of concurrent and parallel programming concepts in Java by applying them to a computationally intensive task, K-Means clustering. Students will explore how various concurrent or parallel programming approaches, such as thread pools, divide-and-conquer, and parallel streams, affect performance. The project also aims to provide students with hands-on experience in designing, implementing, and analysing concurrent/parallel algorithms in a practical context.

1. Students shall work in a team of 5 members to plan and implement the project.
2. Each student is expected to spend 15 hours throughout the semester (approximately 1 to 1.5 hours per week) on this project.
3. Effective planning and time management are crucial to avoid last-minute stress at the end of semester.

C. Project Description

1. Study and Design:
 - a) Study the sequential algorithm(s) for standard K-Means clustering.
 - b) Identify opportunities for concurrency and parallelism in the clustering process (e.g., distance calculations and centroid updates).
2. Implementation:
 - a) Implement three versions of the K-Means algorithm in Java:
 - i. Version 1: Sequential implementation (1) Zhen Hau
 - ii. Version 2: Concurrent implementation using threads (manual thread creation and coordination) (2) Wen Yang, Shao Xian

- iii. Version 3: Parallel implementation using Java Concurrency Framework (e.g., ExecutorService, ForkJoinPool, or parallelStream). (2) YingQi, Haw Yee

Software architect: (one of the person from implementation 2 or 3) YingQi

- Set technical standards (make sure every implementation can be applied to the application)
- Verify and validate implementations and application

3. Performance Evaluation: (same person as sequential) Zhen Hau

- a) Write a Java application to:
 - i. Generate random datasets of different sizes (e.g., 10,000 points, 100,000 points, 1,000,000 points with varying dimensions and values of K).
 - ii. Cluster them using all three implementations (Versions 1 to 3).
 - iii. Measure execution time, memory usage, or other metrics. You may use performance profiling tools, e.g., JvisualVM.
 - iv. Run the application on different hardware setups (if possible, e.g. on the laptops of different team members) or simulate different thread pool sizes.

4. Analysis and Reporting:

- a) Analyse the performance differences across implementations and hardware setups.
- b) Discuss:
 - i. Trade-offs between simplicity, performance, and scalability.
 - ii. Thread overhead vs. speedup.
 - iii. How different concurrency techniques perform under different dataset sizes, dimensions, and hardware setups.
- b) Present your findings in a live demonstration and project report.

Flow:

- Everyone share their findings on sequential implementation
- Software architect selects one of them and oversees the development
- Create GitHub repo (team leader)
- Everyone implements their part of K-means
- Tester develops application to evaluate performance
- Meeting again to discuss and analyse
- Write report

D. Deliverables

1. Live demonstration of 10 to 12 minutes (Week 13) – 10%

- a) Explain the sequential, concurrent and parallel algorithms designed and implemented.
- b) Demonstrate that the developed applications run correctly for the clustering of datasets of different sizes.

- c) Explain the design of experiments to compare the performances of the three implementations in accomplishing the task under different scenarios, e.g. execution time, memory usage, etc.
- d) Compare and discuss the performances of the three implementations in accomplishing the task under different scenarios.
- e) Plan your demonstration so that the above can be clearly demonstrated within the given time limit.
- f) All members must take part in the demonstration.

Submission due: 12pm, Friday 12 June 2026

2. Written report of 10 to 15 pages (Week 14) – 10%

- a) Introduction - Describe K-Means clustering, and its relationship with artificial intelligence and concurrent/parallel programming.
- b) Problem statement – Challenges of sequential algorithms in K-Means clustering and the significance of overcoming the challenges.
- c) Objectives – Relate to the problem to solve, or challenges to overcome.
- d) Methodology – Approaches to achieve the objectives, including data generation, preparation of experimental environment, performance monitoring and analysis.
- e) Design and implementation – Details of the experimental design and execution.
- f) Results and discussions – Present and discuss the results, which may include but not limited to performance graphs and tables, and the corresponding analyses and discussions.
- g) Team reflection and lessons learned.
- h) Appendices - This may include but not limited to source code for all three implementations, complete measurement data..

Submission due: 12pm, Friday 19 June 2026

E. Group Formation and Execution

1. Each group shall consist of 5 members. Application with a valid justification must be made to the course lecturer if a group is to have less/more than 5 members; subject to the course lecturer's approval.
2. The team leader shall register for the whole group via this form: <https://shorturl.at/KRM8J> before 27 March 2026.
3. The team must have a clear structure with responsibilities and accountabilities assigned to each member. Possible project team roles include project manager/team leader, software architect, lead developer, developer, and tester. Each group may have some or all of these roles. Additional roles are allowed if necessary for the project.

4. A member must not do only documentation tasks.
5. Roles, responsibilities, tasks assigned and completed must be clearly indicated in the project report.
6. Use online collaboration tools (Github, Trello, etc.) for collaborative development.
7. Assessments will be made in Weeks 13 (live demonstration) and 14 (written report).

F. Assessments:

Criteria	Excellent	Good	Average	Poor	Very Poor
Week 13 (Live demonstration – 10%)					
Clarity of explaining the K-Means clustering concept in an easy to understand manner.	1.0	0.75	0.5	0.25	0
Correctness and clarity of explanation of the design and implementation of the sequential algorithm.	1.0	0.75	0.5	0.25	0
Correctness and clarity of explanation of the design and implementation of the concurrent algorithm.	1.0	0.75	0.5	0.25	0
Correctness and clarity of explanation of the design and implementation of parallel algorithm.	1.0	0.75	0.5	0.25	0
Correctness of the developed application using the sequential algorithm.	1.0	0.75	0.5	0.25	0
Correctness of the developed application using the concurrent algorithm.	1.0	0.75	0.5	0.25	0
Correctness of the developed application using the parallel algorithm.	1.0	0.75	0.5	0.25	0
Robustness of the experimental design to compare the performances of the	2.0	1.5	1.0	0.5	0

three implementations in accomplishing the task under different scenarios.					
Visualise comparison and discussion of the performances of the three implementations in accomplishing the task under different scenarios	1.0	0.75	0.5	0.25	0

Criteria	Excellent	Good	Average	Poor	Very Poor
Week 14 (Written Report - 10%)					
Introduction - Introduce relevant concepts such as K-Means clustering and its relationships with AI and concurrency/parallelism, etc. in adequate details and length.	1.0	0.75	0.5	0.25	0
Problem statement - Appropriately define the problem that justifies the need for this group project.	1.0	0.75	0.5	0.25	0
Objectives - Specific, measurable, attainable, relevant, and time-bound.	1.0	0.75	0.5	0.25	0
Methodology - Following the methodology should lead to achievement of the objectives and answering the problem.	1.0	0.75	0.5	0.25	0
Design and implementation - Each version of the algorithm design and implementation is correct and clearly explained.	3.0	2.25	1.5	0.75	0
Results and discussions - Results are clearly shown and discussions answer the problem.	2.0	1.5	1.0	0.5	0
Appendices - Relevant additional information such as raw data, code snippets, etc. available to provide a	1.0	0.75	0.5	0.25	0

more comprehensive understanding of the study/results.

--	--	--	--	--

G. Peer Evaluation

A peer evaluation mechanism will be used to determine each group member's contribution factor. The contribution factor will be multiplied with the group marks obtained to produce your final marks. Every group member must submit his/her peer evaluation and evaluate every member in the group, including himself/herself. A peer evaluation form will be created and announced later for each student to fill in at the end of the group project.

The contribution factor will be computed for each group member based on the formula stipulated below:

Peer Evaluation Score	Contribution Factor
0-4	0
5-8	0.2
9-12	0.4
13-16	0.6
17-20	0.8
21-24	1.0

Final contribution factor for each student will be the mode of all contribution factors given by his/her peers, including himself/herself. Critical disputes in assigning peer evaluation scores will be investigated by the course lecturer.

The final marks for each student will be computed using the following formula:

Individual final marks = (Presentation group marks + Report group marks) * individual contribution factor

For example, if a group obtains 8 marks and 7 marks respectively for Presentation and Report, and the contribution factor for a student is 0.8, the student's final marks will be:

$$(8 + 7) * 0.8 = 12 / 20$$

H. Academic Integrity & AI Usage Policy

This assignment operates under Level 2 (limited usage – specific tasks only) of the Universiti Malaya's policy and guidelines for use of AI in academic and research settings.

You are permitted and encouraged to use Generative AI tools (e.g., ChatGPT, Gemini, Grok, Claude, Codex, Antigravity, etc.) to assist with brainstorming, structuring your matrix, refining your academic tone, refining your source code, or summarizing complex technical jargon. However, your use of AI must adhere to the following strict conditions:

1. **Significant and Novel Human Contribution:** The core intellectual work, specifically the design of parallel algorithms, the strategies used for thread synchronization, the resolution of race conditions or deadlocks, and the analysis of system performance, must be your own original thinking. AI cannot replace your problem-solving and debugging skills as an undergraduate software engineering student. Your submission must demonstrate a deep, practical understanding of concurrent programming concepts that goes beyond code an AI can generate out-of-the-box.
2. **Mandatory AI Usage Log:** You must attach an "AI Usage Log" as an appendix to your final submission (e.g., included in your project report or as a README file in your code repository). This log serves as transparent evidence of your AI involvement. It must clearly detail:
 - a) Which AI tool(s) were used.
 - b) The specific prompts or queries inputted.
 - c) How the AI's output (e.g., code snippets, debugging advice, algorithmic logic) was utilized, tested, and subsequently modified in your final implementation.
3. **Penalties for Policy Violation:** Failure to attach an accurate AI usage log, passing off AI-generated code, architecture, or performance analysis as your own original work, or otherwise violating UM's AI use policy and guidelines will result in substantial mark deductions and potential academic disciplinary action.
4. **Appendix A** is a standardized AI Usage Log template. You can include this as an appendix of your report submission. It allows you to reflect on how you used the AI, ensuring you retain intellectual ownership of the submitted work.

I. Late Submission

If for some good reason you are unable to submit your deliverables by the deadline, email the course lecturer to seek a late submission. This should be done prior to the deadline. Late submission without prior approval will have your marks deducted (5% per day overdue), or zero marks be assigned 3 days after the deadline.

J. Penalties

If you plagiarize someone else' work, or violates the AI usage conditions, one or more of the following penalties may be imposed:

1. Loss of all or partial marks for the assessment items;
2. Downgrading the final grade in the course;
3. Imposing a grade of failure in the course.

-END-

Appendix A: Mandatory AI Usage Log**Student Name:** _____**Student ID:** _____**Course Code:** _____**Level of AI Usage:** 2 _____**Instructions:**

In accordance with Universiti Malaya's Level 2 AI Usage Policy, you must transparently document any use of Generative AI tools (e.g., ChatGPT, Claude, Gemini, Elicit, etc.) used in the preparation of this assignment.

Your log must clearly demonstrate that your core intellectual contribution (e.g., algorithmic design, thread synchronization logic, performance tuning, and debugging) remains significant, novel, and entirely your own.

Date	AI Tool Used	Task / Purpose	Specific Prompt(s) Used	Summary of AI Output	How the Output was Used, Critiqued, or Modified in the Final Report

Example: 12 March 2026	ChatGPT 4.0	Understanding synchronization concepts.	"Explain how to implement a reader-writer lock using threads in Java to avoid writer starvation."	Provided an explanation of ReadWriteLock and a basic code snippet demonstrating its use.	I reviewed the logic to understand the fairness policy. I did not copy the code directly; instead, I applied the concept to design my own SharedDataBuffer class tailored to my project's architecture.
Example: 15 March 2026	Gemini 2.5	Debugging a race condition	"Why is my Java ExecutorService K-means clustering code producing inconsistent results? [Pasted 20 lines of my Java code]"	Identified that a shared primitive variable totalSum was being updated by multiple threads simultaneously without synchronization	I updated my code to use an AtomicInteger and the addAndGet() method, which resolved the race condition without the heavy overhead of a synchronized block. I then independently benchmarked the performance.

Student Declaration:

By submitting this log, I confirm that the information provided is accurate and that the critical analysis, synthesis, and final conclusions presented in this assignment are entirely my own original intellectual work. I understand that misrepresenting my use of AI constitutes an academic offense.

Signature: _____ Date: _____